Estructura Unión-Búsqueda (Conjuntos Disjuntos)

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2025

Jorge Mario Londoño Peláez & Varias AI

August 29, 2025

Estructura Unión-Búsqueda (Conjuntos Disjuntos)

La estructura Union-Búsqueda y sus aplicaciones para representar relaciones de conectividad y conjuntos disyuntos.

Contents

1 El problema de componentes conexas / Conjuntos disjuntos **2**

2 API y representación **2**

3 Implementación de la estructura Union-Find **3**

3.1 Búsqueda rápida: QuickFind 3

3.2 Representación por árboles: QuickUnion 3

3.3 Unión ponderada: WeightedQuickUnion 4

4 Optimizaciones y Análisis de Eficiencia **4**

4.1 Compresión de Caminos 4

4.2 Análisis amortizado 4

5 Ejercicios **5**

1 El problema de componentes conexas / Conjuntos disjuntos

El problema de componentes conexas (o conjuntos disjuntos) se centra en determinar la conectividad entre diferentes nodos en un grafo o red. Dos nodos se consideran conectados si existe un camino entre ellos. Esta relación de conectividad es una relación de equivalencia, lo que significa que cumple con las siguientes propiedades:

- **Reflexiva:** Todo nodo está conectado consigo mismo.
- **Simétrica:** Si el nodo A está conectado al nodo B, entonces el nodo B está conectado al nodo A.
- **Transitiva:** Si el nodo A está conectado al nodo B, y el nodo B está conectado al nodo C, entonces el nodo A está conectado al nodo C.

El objetivo principal es, dado un conjunto de nodos y un conjunto de conexiones entre ellos, poder determinar eficientemente si dos nodos dados están conectados y, en caso de que no lo estén, unirlos.

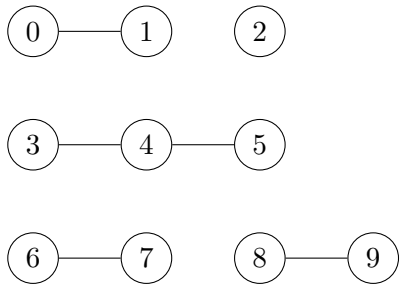

Figure 1: Un grafo con 10 nodos y 5 componentes conexas: {0, 1}, {2}, {3, 4, 5}, {6, 7}, {8, 9}.

2 API y representación

La estructura Union-Find representa un conjunto de n elementos distintos, cada uno representando un conjunto disjunto. Convencionalmente, los elementos se representan con números naturales de 0 a $n-1$. Para gestionar estos conjuntos, el API básico consta de las siguientes operaciones:

- **find(p):** Determina el **representante** del conjunto al que pertenece el elemento p. Dos elementos p y q están en el mismo conjunto si y solo si ‘find(p) == find(q)‘.
- **connected(p, q):** Verifica si los elementos p y q están en el mismo conjunto. Es una operación de conveniencia equivalente a comparar los resultados de ‘find(p)‘ y ‘find(q)‘.
- **union(p, q):** Une los conjuntos que contienen a los elementos p y q. Si ya estaban en el mismo conjunto, no hace nada.
- **count():** Devuelve el número de conjuntos disjuntos.

3 Implementación de la estructura Union-Find

3.1 Búsqueda rápida: QuickFind

La implementación **QuickFind** utiliza un arreglo ‘id[]‘ de tamaño n, donde ‘id[i]‘ es el identificador del componente al que pertenece el elemento i. Inicialmente, cada elemento está en su propio conjunto, por lo que ‘id[i] = i‘ para todo i.

Algorithm 1 QuickFind

```
Estructura: Arreglo ‘id[]‘ de tamaño n.
Inicialización: ‘id[i] = i‘ para i = 0, ..., n − 1.
procedure FIND(p)
    return ‘id[p]‘
end procedure
procedure UNION(p, q)
    p_id ← FIND(p)
    q_id ← FIND(q)
    if p_id ≠ q_id then
        for i ← 0 to n − 1 do
            if id[i] = p_id then
                id[i] ← q_id
            end if
        end for
    end if
end procedure
```

QuickFind ofrece una operación ‘find()‘ muy rápida ($O(1)$), pero la operación ‘union()‘ es lenta ($O(n)$) porque debe recorrer todo el arreglo. Esto la hace ineficiente para problemas con un gran número de uniones.

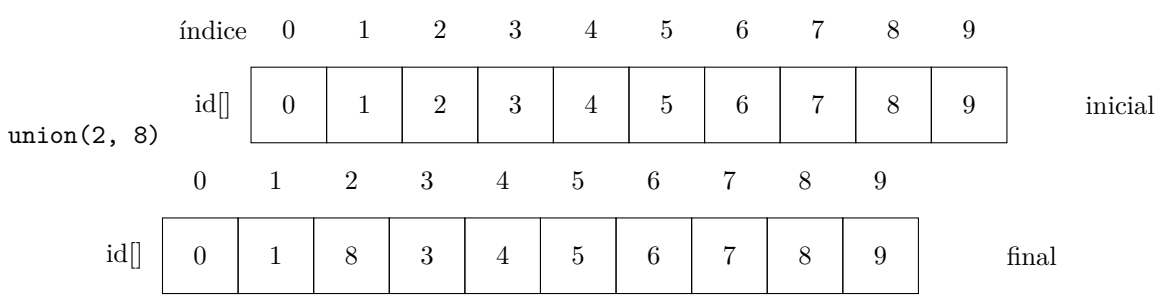

Figure 2: En QuickFind, para unir 2 y 8, se cambia el id de todos los elementos con id 2 al id 8. En este caso, solo se cambia ‘id[2]‘.

3.2 Representación por árboles: QuickUnion

La implementación **QuickUnion** utiliza un arreglo ‘parent[]‘ de tamaño n, donde ‘parent[i]‘ es el padre del elemento i. Cada conjunto se representa como un árbol, y la raíz del árbol es el representante del conjunto. Inicialmente, cada elemento es su propio padre (‘parent[i] = i‘).

Algorithm 2 QuickUnion

```
Estructura: Arreglo ‘parent[]‘ de tamaño n.
Inicialización: ‘parent[i] = i‘ para i = 0, ..., n − 1.
procedure FIND(p)
    while p ≠ parent[p] do
        p ← parent[p]
    end while
    return p
end procedure
procedure UNION(p, q)
    root_p ← FIND(p)
    root_q ← FIND(q)
    if root_p ≠ root_q then
        parent[root_p] ← root_q
    end if
end procedure
```

Aunque la operación `union` parece más rápida que en QuickFind, depende del tiempo de la operación `find` y esta a su vez ser lenta si los árboles se vuelven muy altos. En el peor caso, un árbol puede degenerar en una lista enlazada, haciendo que `find` tome tiempo $O(n)$. Aunque la complejidad del peor caso no mejora, esta estructura arbórea es la base para optimizaciones mucho más eficientes.

3.3 Unión ponderada: WeightedQuickUnion

Esta es una mejora de QuickUnion que evita la formación de árboles altos. Se usa un arreglo adicional ‘size[]‘, donde ‘size[i]‘ almacena el número de nodos en el árbol con raíz i. En una operación ‘union‘, siempre se enlaza la raíz del árbol más pequeño a la raíz del árbol más grande.

Esta simple heurística asegura que la altura de los árboles sea como máximo $O(\log n)$. Como resultado, las operaciones ‘find‘ y ‘union‘ tienen una complejidad de $O(\log n)$.

4 Optimizaciones y Análisis de Eficiencia

4.1 Compresión de Caminos

La compresión de caminos es una optimización adicional para QuickUnion (y sus variantes) que aplana la estructura del árbol durante las operaciones ‘find‘. Cuando se busca la raíz de un nodo ‘p‘, se hace que todos los nodos en el camino desde ‘p‘ hasta la raíz apunten directamente a la raíz.

Con la unión ponderada y la compresión de caminos, la eficiencia de la estructura de datos mejora drásticamente.

4.2 Análisis amortizado

El **análisis amortizado** se usa para analizar secuencias de operaciones. En lugar de acotar el costo de la peor operación individual, se promedia el costo sobre una secuencia de operaciones.

Para Union-Find con unión ponderada y compresión de caminos, se ha demostrado que cualquier secuencia de m operaciones sobre n elementos toma un tiempo casi lineal, del orden de $O(m \cdot \alpha(n))$. La función $\alpha(n)$ es la función inversa de Ackermann, que crece de forma extremadamente lenta.

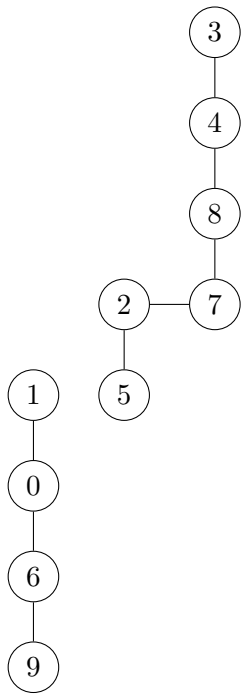

Figure 3: Peor caso en QuickUnion. Uniones como ‘union(9,6)‘, ‘union(6,0)‘, ‘union(0,1)‘ pueden crear árboles altos y delgados, llevando la complejidad de ‘find‘ a $O(n)$.

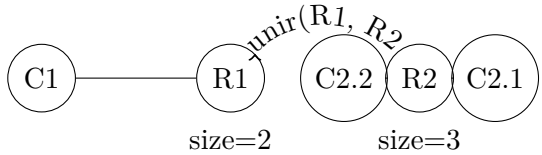

Figure 4: En WeightedQuickUnion, al unir dos árboles, el más pequeño (raíz R1, tamaño 2) se enlaza al más grande (raíz R2, tamaño 3), manteniendo el árbol resultante más chato.

Para cualquier valor práctico de n (incluso más grande que el número de átomos en el universo), $\alpha(n)$ es menor que 5. Por lo tanto, para fines prácticos, se puede considerar que el costo amortizado por operación es casi constante.

5 Ejercicios

1. Considere un conjunto de 10 elementos $0, 1, \ldots, 9$. Muestre el contenido del arreglo ‘parent[]‘ después de la siguiente secuencia de uniones usando (a) QuickUnion y (b) WeightedQuickUnion: ‘union(4, 3), union(3, 8), union(6, 5), union(9, 4), union(2, 1), union(5, 0), union(7, 2), union(6, 1), union(7, 3)‘

2. ¿Cuál es la altura máxima de un árbol en una estructura WeightedQuickUnion con N elementos? Explique su razonamiento.

3. Describa una secuencia de $N - 1$ operaciones de unión sobre N elementos que resulte en un árbol de altura $N - 1$ (el peor caso) para la implementación QuickUnion.

Algorithm 3 WeightedQuickUnion

```
Estructura: Arreglos ‘parent[]‘ y ‘size[]‘ de tamaño n.
Inicialización: ‘parent[i] = i‘, ‘size[i] = 1‘ para i = 0, ..., n − 1.
procedure FIND(p)
    while p ≠ parent[p] do
        p ← parent[p]
    end while
    return p
end procedure
procedure UNION(p, q)
    root_p ← FIND(p)
    root_q ← FIND(q)
    if root_p ≠ root_q then
        if size[root_p] < size[root_q] then
            parent[root_p] ← root_q
            size[root_q] ← size[root_q] + size[root_p]
        else
            parent[root_q] ← root_p
            size[root_p] ← size[root_p] + size[root_q]
        end if
    end if
end procedure
```

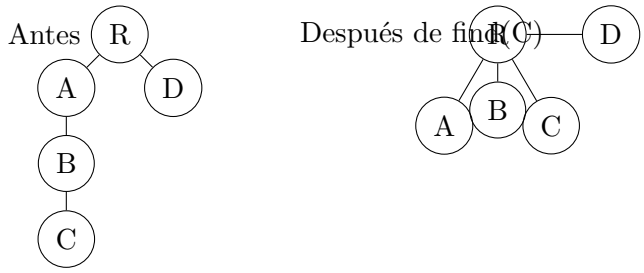

Figure 5: Efecto de la compresión de caminos al ejecutar ‘find(C)‘. Los nodos A, B y C ahora apuntan directamente a la raíz R.

Bibliografía

[1] Aditya Bhargava. *Grokking Algorithms*. Manning Publications, 2016.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009.

[3] Narasimha Karumanchi. *Data Structures and Algorithms Made Easy*. CareerMonk Publications, 2011.

[4] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Pearson, 2005.

[5] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley Professional, 4th edition, 2011.